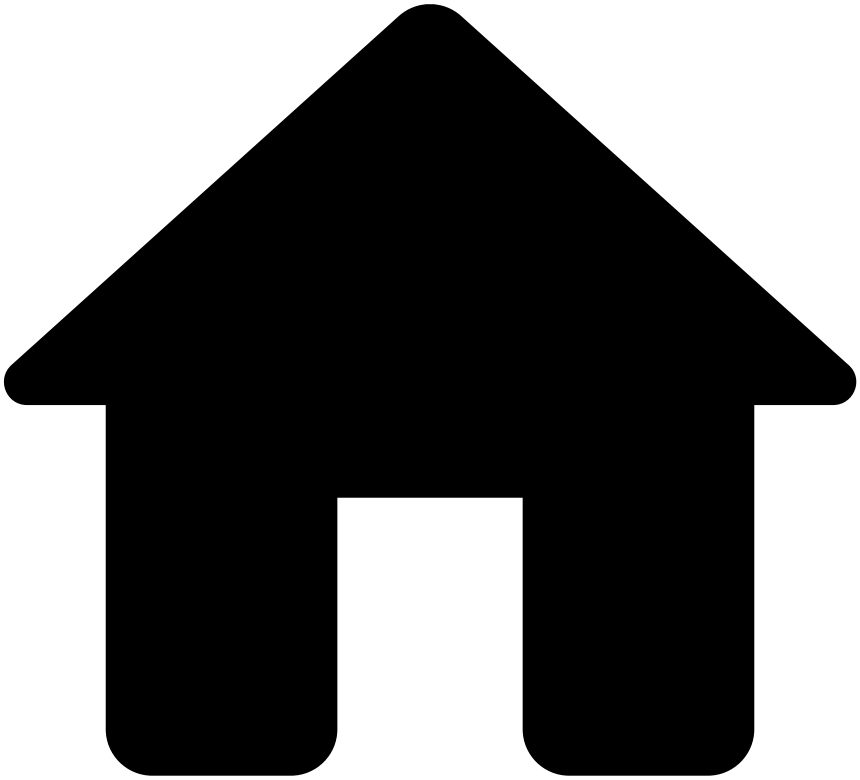


LightGBM Logging Configuration

The LightGBM provider of Pulse can be configured to log information. You can configure the logging levels, from the most basic INFO, up to the most extreme level TRACE on the several modules of the LightGBM provider.

This topic describes the logging levels, and how to set up logging.

Logging levelsâ

There are several modules available, each with different responsibilities. To fill out the logging configurations you must use the fully qualified name of the module (as in the examples on the bottom of the page), by adding the prefix "com.feedzai.openml.provider.lightgbm." to the module name. You can control the logging level per module.

Module name	Description of the module	Description of the logging levels
LightGBMBinaryClassificationModel	Implements the OpenML class responsible for loading a model and scoring instances. It is also called when saving the model to disk.	ERROR: Report that saving model to disk failed.
LightGBMBinaryClassificationModelTrainer	Responsible for training a LightGBM model. At the end of the train it saves an intermediate representation to disk that is then passed to LightGBMBinaryClassificationModel for the final model save to disk.	INFO: Information messages reporting the stage at which the training is ERROR: For errors during train DEBUG: Report #train-instances, training parameters and features, more details on the sub-stages of training, progress in number of train iterations
LightGBMModelCreator	Starts all load/fit operations and validations prior to starting the aforementioned model load/fit process.	INFO: Message with file path from which the model is being loaded ERROR: Errors when training the model or the validations for a model fit/load fail and where it is it trying to read from.
LightGBMSWIG	Low-level implementation for model scoring.	INFO: Saving model to disk calls. ERROR: Errors saving model to disk. DEBUG: Saving model to disk calls with filepaths. TRACE: Show prediction score per instance (**might be too much logging!**)
LightGBMUtils	Loads the LightGBM libraries.	INFO: When the libraries are loaded. DEBUG: Names of the libraries being loaded.
SWIGResources	Lowest-level implementation level for resources handling during model scoring.	DEBUG: In-memory instantiation of model. TRACE: Resource release calls.
SWIGTrainResources	Lowest-level implementation level for resources handling during model fitting.	DEBUG: Details for train memory allocation.

How to set up loggingâ

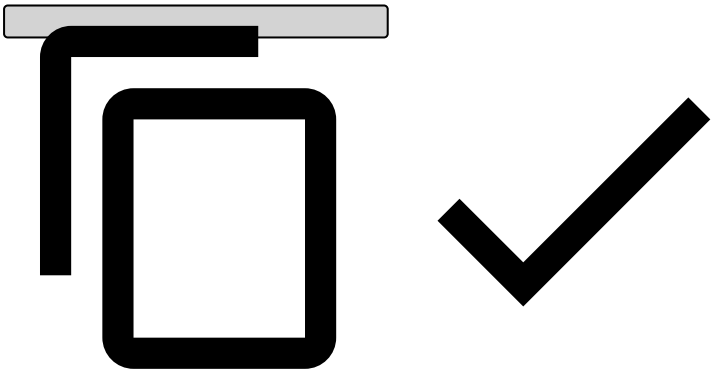
To configure such log levels, edit the file `conf/logger.xml`. Also, for Spark Standalone `spark-libs/logback-executors.xml` should also have such log levels.

Example configurationâ

The following snippet is an example of configuring the logging level for all modules of LightGBM. Debug is a good default level.

In files `conf/logger.xml` and `spark-libs/logback-executors.xml`:

```
 <logger name="com.feedzai.openml.provider.lightgbm" level="debug"/>
```



Alternatively, the debug level can be chosen/overriden individually for each component. The snippet below is equivalent to the snippet above, all components use the debug level:

In files `conf/logger.xml` and `spark-libs/logback-executors.xml`:

```
 <logger name="com.feedzai.openml.provider.lightgbm.LightGBMBinaryClassificationModel" level="deb
ug"/>
<logger name="com.feedzai.openml.provider.lightgbm.LightGBMBinaryClassificationModelTrainer"
level="debug"/>
<logger name="com.feedzai.openml.provider.lightgbm.LightGBMModelCreator" level="debug"/>
<logger name="com.feedzai.openml.provider.lightgbm.LightGBMSWIG" level="debug"/>
<logger name="com.feedzai.openml.provider.lightgbm.LightGBMUtils" level="debug"/>
<logger name="com.feedzai.openml.provider.lightgbm.SWIGResources" level="debug"/>
<logger name="com.feedzai.openml.provider.lightgbm.SWIGTrainResources" level="debug"/>
```